Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №3
по дисциплине
«Современные системы программирования»

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды разработки 5
2.4 Пример инспекции типа 5
2.5 Пример работы с приватными членами 6
2.6 Пример позднего связывания 7
2.7 Задание 1. Класс с приватными полями и свойствами с валидацией 9
2.8 Задание 2. Информация о методах класса 9
2.9 Задание 3. Создание экземпляра через *Activator.CreateInstance* 10
2.10 Задание 4. Класс «черный ящик» 11
2.11 Задание 5. Доступ к приватному полю и его изменение 11
2.12 Задание 6. Вызов приватного метода 12
2.13 Задание 7. Система динамической загрузки плагинов 12
3 Ответы на контрольные вопросы 14
Выводы 16
Приложение А (обязательное) Листинги программного кода 17

1 ЦЕЛЬ РАБОТЫ

Изучить механизм рефлексии платформы *.NET* и позднее связывание в языке *C#*: научиться получать метаданные о типах через *System.Type*, инспектировать конструкторы, методы, свойства и поля, создавать объекты без оператора *new*, обращаться к приватным полям и методам, а также построить расширяемую систему плагинов, загружаемых по имени из конфигурационного файла.

2 ХОД РАБОТЫ

Работа выполнена в консольном приложении *C#* для платформы *.NET* 10 (*SDK* 10.0.401). Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для изучения рефлексии в рамках работы требуется выполнить следующие задачи:

1 создать класс с тремя приватными полями разных типов и публичными свойствами с проверкой значений;

2 получить и вывести сведения о публичных и непубличных методах класса;

3 создать экземпляры класса через *Activator.CreateInstance* конструкторами без параметров и с параметрами;

4 создать класс «черный ящик» и получить доступ к его приватному полю и приватным методам через рефлексию;

5 реализовать систему динамической загрузки плагинов по атрибуту, интерфейсу и имени типа из файла конфигурации.

2.2 Теоретические сведения

Рефлексия – механизм, позволяющий получать информацию о типах данных во время выполнения программы и динамически работать с ними. Основное пространство имен – *System.Reflection*, основные классы – *Type*, *FieldInfo*, *PropertyInfo*, *MethodInfo*, *ConstructorInfo*, *Assembly*, *Activator*.

Объект *Type* получают тремя способами: оператором *typeof*(*T*) для типа, известного при компиляции, методом *GetType*() для существующего объекта и методом *Type.GetType*("имя") по строковому имени типа. Члены типа возвращают методы *GetFields*(), *GetMethods*(), *GetProperties*(), *GetConstructors*(). Набор искомых членов задается флагами *BindingFlags*: *Public*, *NonPublic*, *Instance*, *Static*.

Позднее связывание (*Late Binding*) – выбор типа, конструктора или метода во время выполнения, а не при компиляции. Динамическое создание объекта выполняет *Activator.CreateInstance*, вызов метода – *MethodInfo.Invoke*. Атрибуты – дополнительные метаданные класса или метода, которые читаются методом *GetCustomAttribute*. Рефлексия позволяет обращаться к приватным членам, поэтому нарушает инкапсуляцию, работает

медленнее прямого вызова, а ошибки обнаруживаются только при выполнении.

2.3 Подготовка среды разработки

Приложение *Lab3* создано как консольный проект *.NET* с целевой платформой *net10.0* (*SDK* 10.0.401). Конфигурационный файл *plugins.config* копируется в каталог сборки и содержит имена загружаемых плагинов. Запуск: *dotnet build -c Release*, затем *dotnet bin\Release\net10.0\Lab3.dll*. Примеры из методических указаний (инспекция типа *Product*, работа с приватными членами, позднее связывание на классе *DynamicCalculator*) вынесены в файл *Examples.cs*, задания 1–7 – в файлы *Tasks.cs* и *Plugins.cs*.

2.4 Пример инспекции типа

Программа получает объект *Type* для класса *Product* и выводит имя, полное имя, пространство имен, сборку, базовый тип, характеристики типа, конструкторы, методы, свойства и поля, включая приватные. Результат показан на рисунках 2.1 и 2.2.

```
> dotnet Lab3.dll
=== ПРИМЕР: БАЗОВАЯ ИНСПЕКЦИЯ ТИПА ===

1.1 Базовая информация:
  Имя типа: Product
  Полное имя: LabReflection.Product
  Пространство имен: LabReflection
  Сборка: Lab3
  Базовый тип: Object

1.2 Характеристики:
  IsClass: True
  IsAbstract: False
  IsSealed: False
  IsPublic: True
  IsValueType: False

1.3 Конструкторы:
  .ctor() [Public: True]
  .ctor(Int32 id, String name, Double price) [Public: True]

1.4 Методы:
  Int32 get_Id()
  Void set_Id(Int32 value)
  String get_Name()
```

Рисунок 2.1 – Инспекция типа *Product* (начало)

Полное имя типа *LabReflection.Product*, сборка *Lab3*, базовый тип *Object*. Тип является открытым классом, не абстрактным и не запечатанным. Выведены два конструктора: без параметров и с тремя параметрами.

Продолжение вывода с методами, свойствами и полями показано на рисунке 2.2.

```
  Void set_Name(String value)
  Double get_Price()
  Void set_Price(Double value)
  String get_Category()
  Void DisplayInfo()
  Double ApplyDiscount(Double percent)
  Int32 GetTotalCount()
  ... и еще 4 методов

1.5 Свойства:
  Int32 Id { getset }
  String Name { getset }
  Double Price { getset }
  String Category { getset }

1.6 Поля:
  private  Int32 _id
  private  String _name
  protected  Double _price
  private  Int32 <Id>k__BackingField
  private  String <Name>k__BackingField
  private  Double <Price>k__BackingField
  private  String <Category>k__BackingField
  public static Int32 TotalProducts
```

Рисунок 2.2 – Инспекция типа *Product* (продолжение)

Первые десять методов включают аксессоры свойств *get_* и *set_*, а также *DisplayInfo* и *ApplyDiscount*. В списке полей кроме объявленных присутствуют автоматические поля свойств вида *<Id>k__BackingField* и статическое поле *TotalProducts*.

2.5 Пример работы с приватными членами

Через *FieldInfo* с флагами *NonPublic* | *Instance* читается и изменяется приватное поле *_price*, вызывается приватный метод *ValidatePrice*, изменяется поле *_name* и вызывается защищенный метод *UpdateCategory*. Результат показан на рисунке 2.3.

```
> dotnet Lab3.dll
=== ПРИМЕР: РАБОТА С ПРИВАТНЫМИ ЧЛЕНАМИ ===

2.1 Работа с приватным полем _price:
  Текущее значение _price: 1000
  Новое значение _price: 1500

2.2 Вызов приватного метода ValidatePrice:
  Цена до валидации: -500
  Цена после валидации: 0

2.3 Работа с приватным полем _name:
  Текущее имя: Laptop
  Новое имя: Gaming Laptop

2.4 Вызов защищенного метода UpdateCategory:
  Категория до: Default
  Категория после: Electronics
```

Рисунок 2.3 – Работа с приватными членами класса *Product*

Значение *_price* изменено с 1000 на 1500. Отрицательная цена минус 500 после вызова приватного метода *ValidatePrice* стала равной 0. Имя изменено с *Laptop* на *Gaming Laptop*, а категория после вызова защищенного метода изменилась с *Default* на *Electronics*.

2.6 Пример позднего связывания

Тип *DynamicCalculator* находится по строковому имени *LabReflection.DynamicCalculator*, экземпляр создается через *Activator.CreateInstance*, методы *Add* с разными параметрами, *Multiply*, *SetResult*, приватный *LogOperation* и статический *GetVersion* вызываются через *MethodInfo.Invoke*, второй экземпляр создается конструктором с параметром. Результат показан на рисунках 2.4 и 2.5.

```
> dotnet Lab3.dll
=== ПРИМЕР: ПОЗДНЕЕ СВЯЗЫВАНИЕ ===

3.1 Динамическое получение типа:
  Тип найден: DynamicCalculator
  Полное имя: LabReflection.DynamicCalculator

3.2 Динамическое создание экземпляра:
  [Конструктор DynamicCalculator без параметров]
  Экземпляр создан: DynamicCalculator

3.3 Динамический вызов методов:
  Add(10, 20) = 30
  Результат Add(10,20): 30
  Add(5.5, 3.2) = 8.7
  Результат Add(5.5,3.2): 8.7

3.4 Вызов с параметрами разного типа:
  Multiply(7, 8) = 56
```

Рисунок 2.4 – Позднее связывание (начало)

Тип найден по имени, объект создан без оператора *new*. Вызовы дали *Add*(10, 20) = 30, *Add*(5,5; 3,2) = 8,7 и *Multiply*(7, 8) = 56, причем нужная перегрузка *Add* выбрана по типам параметров.

Продолжение вывода программы показано на рисунке 2.5.

```
  Результат Multiply(7,8): 56

3.5 Работа со свойствами через рефлексию:
  Текущий Result: 0
  Установлен результат: 999
  Result после SetResult: 999

3.6 Вызов приватного метода:
  [LOG] Выполнена операция: Тестовая операция
  Приватный метод LogOperation вызван успешно

3.7 Вызов статического метода:
  Версия: DynamicCalculator v1.0

3.8 Создание экземпляра через конструктор с параметром:
  [Конструктор с параметром: начальное значение = 42]
  Экземпляр с параметром создан
  Начальное значение Result: 42
```

Рисунок 2.5 – Позднее связывание (продолжение)

Свойство *Result* после *SetResult*(999) равно 999. Приватный метод *LogOperation* и статический метод *GetVersion* вызваны успешно, версия *DynamicCalculator v1.0*. Конструктор с параметром 42 установил начальное значение *Result*, равное 42.

2.7 Задание 1. Класс с приватными полями и свойствами с валидацией

Класс *ReflectionTarget* содержит три приватных поля разных типов: *_age* (*int*), *_name* (*string*), *_rating* (*double*). Для каждого поля есть публичное свойство с проверкой: возраст не может быть меньше нуля, пустое имя заменяется на *Unknown*, рейтинг ограничивается диапазоном от 0 до 5. Код приведен в приложении А.

Проверка валидации показана на рисунке 2.6.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 1: КЛАСС С ПРИВАТНЫМИ ПОЛЯМИ И СВОЙСТВАМИ ===

После Age=-5, Name="", Rating=10:
  Age=0, Name=Unknown, Rating=5
После Age=20, Name="Anna", Rating=4.5:
  Age=20, Name=Anna, Rating=4.5
```

Рисунок 2.6 – Проверка валидации свойств

Значения возраста минус 5, пустого имени и рейтинга 10 заменены на 0, *Unknown* и 5. Допустимые значения 20, *Anna* и 4,5 сохранены без изменений.

2.8 Задание 2. Информация о методах класса

Метод *GetMethods* вызывается с флагами *Public* | *NonPublic* | *Instance* | *Static*. Для каждого метода выводятся модификатор доступа, статичность, возвращаемый тип, имя и параметры, а методы *get_* и *set_*, созданные для свойств, отфильтровываются. Для наглядности использован класс *DynamicCalculator*, содержащий публичные, приватный, статический и экземплярные методы. Код приведен в приложении А.

Результат показан на рисунке 2.7.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 2: МЕТОДЫ КЛАССА ЧЕРЕЗ РЕФЛЕКСИЮ ===

Все методы (BindingFlags.Public | NonPublic | Instance | Static):
  public Int32 Add(Int32 a, Int32 b)
  public Double Add(Double a, Double b)
  public Int32 Multiply(Int32 a, Int32 b)
  private Void LogOperation(String operation)
  public Void SetResult(Int32 value)
  public static String GetVersion()
  public Type GetType()
  protected Object MemberwiseClone()
  protected Void Finalize()
  public String ToString()
  public Boolean Equals(Object obj)
  public Int32 GetHashCode()

Разница между флагами привязки (с учётом get_/set_ и методов object):
  Public | Instance:    9 методов
  NonPublic | Instance: 3 методов
  Public | Static:     1 методов
```

Рисунок 2.7 – Методы класса *DynamicCalculator*

Вывод содержит публичные перегрузки *Add*, *Multiply*, *SetResult*, приватный *LogOperation*, статический *GetVersion* и унаследованные методы *object*, включая защищенные *MemberwiseClone* и *Finalize*. Флаги дают разные наборы: *Public* | *Instance* – 9 методов, *NonPublic* | *Instance* – 3 метода, *Public* | *Static* – 1 метод.

2.9 Задание 3. Создание экземпляра через *Activator.CreateInstance*

Объект создается без оператора *new* двумя способами: конструктором без параметров и конструктором с параметрами. Для первого объекта значения свойств задаются и читаются через *PropertyInfo*. Код приведен в приложении А.

Результат показан на рисунке 2.8.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 3: Activator.CreateInstance ===

Создан объект конструктором без параметров.
  Age = 30
  Name = Reflection object
  Rating = 4
Создан объект конструктором с параметрами (25, "With params", 3.5).
  Age = 25
  Name = With params
  Rating = 3.5
```

Рисунок 2.8 – Создание объектов через *Activator.CreateInstance*

Первый объект после установки значений через рефлексию содержит возраст 30, имя *Reflection object* и рейтинг 4. Второй объект, созданный с аргументами (25, *With params*, 3,5), содержит те же значения, что были переданы.

2.10 Задание 4. Класс «черный ящик»

Класс *SecretCalculator* содержит приватное поле *_secretValue*, публичный метод *ReadValue*() для проверки значения и приватные методы *Calculate*(*int a*, *int b*) и *Reset*(). Прямого доступа к полю и методам извне нет. Код приведен в приложении А.

Начальное состояние объекта показано на рисунке 2.9.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 4: ЧЁРНЫЙ ЯЩИК SecretCalculator ===

ReadValue() = 10
Поле _secretValue и метод Calculate недоступны извне (private).
```

Рисунок 2.9 – Начальное состояние класса *SecretCalculator*

Публичный метод *ReadValue*() возвращает 10. Обратиться к *_secretValue* и *Calculate* из внешнего кода обычным способом нельзя, они доступны только через рефлексию.

2.11 Задание 5. Доступ к приватному полю и его изменение

Поле *_secretValue* ищется методом *GetField* с флагами *NonPublic* | *Instance*, значение читается и записывается через *FieldInfo*. Результат проверяется публичным методом *ReadValue*(). Отсутствие поля и нехватка прав обрабатываются исключениями *MissingFieldException* и *FieldAccessException*. Код приведен в приложении А.

Результат показан на рисунке 2.10.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 5: ИЗМЕНЕНИЕ ПРИВАТНОГО ПОЛЯ ===

Значение до изменения: 10
Значение через публичный метод ReadValue() после SetValue: 25
Рефлексия нарушила инкапсуляцию: приватное поле изменено.
Ошибка: Поле _unknown не найдено
```

Рисунок 2.10 – Изменение приватного поля через рефлексию

Значение поля изменено с 10 на 25, и публичный метод подтвердил новое значение, то есть рефлексия нарушила инкапсуляцию. Поиск несуществующего поля *_unknown* привел к исключению, которое обработано сообщением об ошибке.

2.12 Задание 6. Вызов приватного метода

Приватный метод *Calculate* находится через *GetMethod* с флагами *NonPublic* | *Instance* и вызывается двумя способами: через *MethodInfo.Invoke* и через делегат *Func<int*, *int*, *int>*, созданный методом *CreateDelegate*. Метод *Reset* без параметров вызывается через *Invoke*. Код приведен в приложении А.

Результат показан на рисунке 2.11.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 6: ВЫЗОВ ПРИВАТНОГО МЕТОДА ===

Способ 1 (Invoke) Calculate(3, 4) = 22
Способ 2 (делегат) Calculate(5, 6) = 40
Перед Reset(): ReadValue() = 10
После Reset() (Invoke без параметров): ReadValue() = 0
```

Рисунок 2.11 – Вызов приватных методов

Вызов *Calculate*(3, 4) через *Invoke* вернул 22 (3 умножить на 4 плюс секретное значение 10), вызов через делегат *Calculate*(5, 6) вернул 40. После *Reset*() метод *ReadValue*() возвращает 0.

2.13 Задание 7. Система динамической загрузки плагинов

Интерфейс *IPlugin* задает контракт плагина, атрибут *PluginAttribute* хранит имя плагина, атрибут *PluginMethodAttribute* описывает метод. Загрузчик читает имена из файла *plugins.config*, получает все типы сборки методом *GetTypes*(), выбирает классы, реализующие *IPlugin*, проверяет атрибут *PluginAttribute*, создает экземпляр через *Activator.CreateInstance* и вызывает методы с атрибутом *PluginMethodAttribute* через *Invoke*. Интерфейс и атрибуты приведены в приложении А.

Код загрузчика плагинов приведен в приложении А.

Для демонстрации гибкости файл *plugins.config* сначала содержит два имени (*MySimplePlugin*, *Logger*), затем в него добавляется имя *Greeter*, а код загрузчика не меняется. Результат показан на рисунках 2.12 и 2.13.

```
> dotnet Lab3.dll
=== ЗАДАНИЕ 7: СИСТЕМА ПЛАГИНОВ ===

Плагины в plugins.config: MySimplePlugin, Logger, Greeter

Загружен плагин: MySimplePlugin (MyPlugin)
  Метод Execute: Основной метод
Привет из плагина!
  Метод Add: Метод для сложения
    Результат Add(5, 3) = 8

Загружен плагин: Logger (LoggerPlugin)
  Метод Execute: Основной метод логгера
[LOG] Logger plugin executed

Загружен плагин: Greeter (GreeterPlugin)
  Метод Execute: Основной метод приветствия
Greeter plugin: добрый день!
```

Рисунок 2.12 – Загрузка плагинов из конфигурации

Загружены плагины *MySimplePlugin* и *Logger*: для каждого выведены описания методов, выполнены методы *Execute*, а для первого плагина метод *Add* вернул 8.

Продолжение вывода программы показано на рисунке 2.13.

```
--- В plugins.config добавлена строка Greeter ---

Плагины в plugins.config: MySimplePlugin, Logger, Greeter, Greeter

Загружен плагин: MySimplePlugin (MyPlugin)
  Метод Execute: Основной метод
Привет из плагина!
  Метод Add: Метод для сложения
    Результат Add(5, 3) = 8

Загружен плагин: Logger (LoggerPlugin)
  Метод Execute: Основной метод логгера
[LOG] Logger plugin executed

Загружен плагин: Greeter (GreeterPlugin)
  Метод Execute: Основной метод приветствия
Greeter plugin: добрый день!
```

Рисунок 2.13 – Загрузка нового плагина без изменения кода

После добавления строки *Greeter* в конфигурацию загрузчик создал и выполнил плагин *GreeterPlugin*. Добавление нового плагина потребовало нового класса и строки в файле конфигурации, а основной код менять не пришлось.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое рефлексия и для чего она используется в *C#*?

Рефлексия – механизм получения информации о типах во время выполнения программы. С ее помощью находят поля, свойства, методы и конструкторы, динамически создают объекты и вызывают методы. Рефлексию используют сериализаторы, *ORM*, *DI*-контейнеры, системы плагинов и тестовые библиотеки.

2. Какие пространства имен и классы необходимы?

Основное пространство имен – *System.Reflection*. Используются классы *Type*, *FieldInfo*, *PropertyInfo*, *MethodInfo*, *ConstructorInfo*, *Assembly* и *Activator*.

3. В чем разница между *GetType*() и *typeof*()?

Оператор *typeof*(*T*) получает тип, известный при компиляции. Метод *GetType*() вызывается у объекта и возвращает его фактический тип во время выполнения.

4. Как получить доступ к приватным членам?

Нужно найти поле или метод методами *GetField*() или *GetMethod*() с флагами *BindingFlags.NonPublic* и *BindingFlags.Instance* (или *Static*), а затем применить *GetValue*, *SetValue* или *Invoke*, как показано в заданиях 5 и 6.

5. Какие ограничения и риски есть у рефлексии?

Рефлексия медленнее прямого обращения, ошибки обнаруживаются только при выполнении, код сложнее сопровождать. Она позволяет обходить инкапсуляцию и случайно изменять приватные данные, а несоответствие реальной структуры типа ожидаемой приводит к исключениям.

6. Что такое *BindingFlags*?

BindingFlags – набор флагов, определяющих, какие члены искать: *Public* (публичные), *NonPublic* (приватные и защищенные), *Instance* (члены экземпляра), *Static* (статические).

7. Как динамически создать экземпляр класса?

Получают объект *Type* и вызывают *Activator.CreateInstance*(*type*). Конструктор с параметрами вызывают передачей аргументов в *Activator.CreateInstance*(*type*, *args*) или через *ConstructorInfo.Invoke*().

8. Чем позднее связывание отличается от раннего?

При раннем связывании типы и методы известны и проверяются компилятором. При позднем связывании тип, конструктор или метод выбираются по имени во время выполнения.

9. Каковы недостатки рефлексии?

Более низкая производительность, отсутствие полной проверки компилятором, ошибки во время выполнения, сложный рефакторинг и

отладка, возможность нарушить инкапсуляцию, необходимость обработки исключений.

ВЫВОДЫ

Изучен механизм рефлексии *C#*: получены метаданные типов, выведены конструкторы, методы, свойства и поля, в том числе приватные. Объекты созданы динамически через *Activator.CreateInstance*, методы вызваны по имени с учетом перегрузок. Приватное поле изменено и приватные методы вызваны через *FieldInfo* и *MethodInfo*, что показало нарушение инкапсуляции. Реализована система плагинов, загружающая классы по имени из конфигурационного файла, причем добавление плагина не потребовало изменения основного кода. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги программного кода

Листинг А.1 – Класс *ReflectionTarget* (задание 1)

```
// Задание 1: класс с тремя приватными полями разных типов и свойствами с валидацией
public class ReflectionTarget
{
    private int _age;
    private string _name;
    private double _rating;

    public ReflectionTarget()
    {
        _name = "Unknown";
    }

    public ReflectionTarget(int  age, string name, double rating)
    {
        Age = age;
        Name = name;
        Rating = rating;
    }

    // Возраст не может быть меньше нуля
    public int Age
    {
        get { return _age; }
        set { _age = value < 0 ? 0 : value; }
    }

    // Пустое имя заменяется на Unknown
    public string Name
    {
        get { return _name; }
        set { _name = string.IsNullOrWhiteSpace(value)  ? "Unknown" : value; }
    }

    // Рейтинг ограничивается диапазоном от 0 до 5
    public double Rating
    {
        get { return _rating; }
        set { _rating = value < 0 ? 0 : (value > 5 ? 5 : value); }
    }
```

Листинг А.2 – Получение информации о методах (задание 2)

```
// Задание 2: информация о всех методах класса
public static void Task2()
{
```

```
    Console.WriteLine("\n=== ЗАДАНИЕ 2: МЕТОДЫ КЛАССА ЧЕРЕЗ РЕФЛЕКСИЮ ===\n");

    // Класс DynamicCalculator содержит публичные, приватные, статические и экземплярные методы
    Type type = typeof(DynamicCalculator);

    // Все методы: публичные, приватные, статические, экземплярные
    MethodInfo[] allMethods = type.GetMethods(
        BindingFlags.Public |
        BindingFlags.NonPublic |
        BindingFlags.Instance |
        BindingFlags.Static);

    Console.WriteLine("Все методы (BindingFlags.Public | NonPublic | Instance | Static):");
    foreach (MethodInfo method in allMethods)
    {
        // Отфильтровываем методы, сгенерированные для свойств
        if (method.Name.StartsWith("get_") || method.Name.StartsWith("set_"))
            continue;

        string access = method.IsPublic ? "public" : (method.IsPrivate ? "private" : "protected");
        string isStatic = method.IsStatic ? " static" : "";
        string parameters = string.Join(", ",
            method.GetParameters().Select(p => $"{p.ParameterType.Name} {p.Name}"));
        Console.WriteLine($"  {access}{isStatic} {method.ReturnType.Name} {method.Name}({parameters})");
    }

    // Разница между флагами привязки
    int publicInstance = type.GetMethods(BindingFlags.Public | BindingFlags.Instance).Length;
    int nonPublicInstance = type.GetMethods(BindingFlags.NonPublic | BindingFlags.Instance).Length;
    int publicStatic = type.GetMethods(BindingFlags.Public | BindingFlags.Static).Length;
    Console.WriteLine("\nРазница между флагами привязки (с учётом get_/set_ и методов object):");
    Console.WriteLine($"  Public | Instance:    {publicInstance} методов");
    Console.WriteLine($"  NonPublic | Instance: {nonPublicInstance} методов");
    Console.WriteLine($"  Public | Static:     {publicStatic} методов");
}
```

Листинг А.3 – Создание экземпляров через *Activator* (задание 3)

```
// Задание 3: создание экземпляра через Activator.CreateInstance
public static void Task3()
{
    Console.WriteLine("\n=== ЗАДАНИЕ 3: Activator.CreateInstance ===\n");

    Type type = typeof(ReflectionTarget);

    // Способ 1: конструктор без параметров
    object target1 = Activator.CreateInstance(type);
    Console.WriteLine("Создан объект конструктором без параметров.");
```

```
    // Установка значений свойств через рефлексию
    type.GetProperty("Age").SetValue(target1, 30);
    type.GetProperty("Name").SetValue(target1, "Reflection object");
    type.GetProperty("Rating").SetValue(target1, 4.0);

    // Чтение значений через рефлексию
    Console.WriteLine($"  Age = {type.GetProperty("Age").GetValue(target1)}");
    Console.WriteLine($"  Name = {type.GetProperty("Name").GetValue(target1)}");
    Console.WriteLine($"  Rating = {type.GetProperty("Rating").GetValue(target1)}");

    // Способ 2: конструктор с параметрами
    object target2 = Activator.CreateInstance(type, new object[] { 25, "With params", 3.5 });
    Console.WriteLine("Создан объект конструктором с параметрами (25, \"With params\",
3.5).");
    Console.WriteLine($"  Age = {type.GetProperty("Age").GetValue(target2)}");
    Console.WriteLine($"  Name = {type.GetProperty("Name").GetValue(target2)}");
    Console.WriteLine($"  Rating = {type.GetProperty("Rating").GetValue(target2)}");
}
```

Листинг А.4 – Класс *SecretCalculator* (задание 4)

```
// Задание 4: "чёрный ящик" с приватным полем и приватным методом
public class SecretCalculator
{
    private int _secretValue = 10;

    // Публичный метод для чтения значения (для проверки)
    public int ReadValue()
    {
        return _secretValue;
    }

    // Приватный метод с логикой вычислений
    private int Calculate(int a, int b)
    {
        return a * b + _secretValue;
    }

    private void Reset()
    {
        _secretValue = 0;
    }
}
```

Листинг А.5 – Изменение приватного поля (задание 5)

```
// Задание 5: доступ к приватному полю и его изменение
public static void Task5()
{
    Console.WriteLine("\n=== ЗАДАНИЕ 5: ИЗМЕНЕНИЕ ПРИВАТНОГО ПОЛЯ ===\n");

    var calculator = new SecretCalculator();
```

```
    Type type = typeof(SecretCalculator);

    try
    {
        FieldInfo field = type.GetField("_secretValue",
            BindingFlags.NonPublic | BindingFlags.Instance);

        if (field == null)
            throw new MissingFieldException("Поле _secretValue не найдено");

        Console.WriteLine($"Значение до изменения: {field.GetValue(calculator)}");
        field.SetValue(calculator, 25);
        Console.WriteLine($"Значение через публичный метод ReadValue() после SetValue:
{calculator.ReadValue()}");
        Console.WriteLine("Рефлексия нарушила инкапсуляцию: приватное поле изменено.");

        // Поле, которого нет в классе
        FieldInfo missing = type.GetField("_unknown",
            BindingFlags.NonPublic | BindingFlags.Instance);
        if (missing == null)
            throw new MissingFieldException("Поле _unknown не найдено");
    }
    catch (MissingFieldException ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
    }
    catch (FieldAccessException ex)
    {
        Console.WriteLine($"Недостаточно прав: {ex.Message}");
    }
}
```

Листинг А.6 – Вызов приватных методов (задание 6)

```
// Задание 6: вызов приватного метода
public static void Task6()
{
    Console.WriteLine("\n=== ЗАДАНИЕ 6: ВЫЗОВ ПРИВАТНОГО МЕТОДА ===\n");

    var calculator = new SecretCalculator();
    Type type = typeof(SecretCalculator);

    MethodInfo calculate = type.GetMethod("Calculate",
        BindingFlags.NonPublic | BindingFlags.Instance);
    MethodInfo reset = type.GetMethod("Reset",
        BindingFlags.NonPublic | BindingFlags.Instance);

    // Способ 1: MethodInfo.Invoke, метод с двумя параметрами
    object result = calculate.Invoke(calculator, new object[] { 3, 4 });
    Console.WriteLine($"Способ 1 (Invoke) Calculate(3, 4) = {result}");

    // Способ 2: делегат
    Func<int, int, int> delegateMethod =
```

```
    (Func<int, int, int>)calculate.CreateDelegate(typeof(Func<int, int, int>), calculator);
  Console.WriteLine($"Способ 2 (делегат) Calculate(5, 6) = {delegateMethod(5, 6)}");

  // Метод без параметров
  Console.WriteLine($"Перед Reset(): ReadValue() = {calculator.ReadValue()}");
  reset.Invoke(calculator, null);
  Console.WriteLine($"После Reset() (Invoke без параметров): ReadValue() =
{calculator.ReadValue()}");
}
```

Листинг А.7 – Интерфейс и атрибуты плагинов

```
[AttributeUsage(AttributeTargets.Class)]
public class PluginAttribute : Attribute
{
  public string Name { get; }
  public PluginAttribute(string name) => Name = name;
}

[AttributeUsage(AttributeTargets.Method)]
public class PluginMethodAttribute : Attribute
{
  public string Description { get; }
  public PluginMethodAttribute(string description) => Description = description;
}

// ============ ИНТЕРФЕЙС ============

public interface IPlugin
{
  void Execute();
}
```

Листинг А.8 – Загрузчик плагинов

```
public static void Load(string configPath)
{
  string[] names = File.ReadAllLines(configPath)
    .Select(line => line.Trim())
    .Where(line => line.Length > 0)
    .ToArray();
  Console.WriteLine($"Плагины в {Path.GetFileName(configPath)}: {string.Join(", ",
names)}");

  // Сборка и все её типы
  Assembly assembly = Assembly.GetExecutingAssembly();
  Type[] types = assembly.GetTypes();

  foreach (Type type in types)
  {
    // Класс должен реализовывать интерфейс IPlugin
    if (!typeof(IPlugin).IsAssignableFrom(type) || type.IsInterface || type.IsAbstract)
      continue;
```

```
        // Атрибут PluginAttribute
        PluginAttribute attribute = type.GetCustomAttribute<PluginAttribute>();
        if (attribute == null || !names.Contains(attribute.Name))
            continue;

        // Создание экземпляра через Activator.CreateInstance
        IPlugin plugin = (IPlugin)Activator.CreateInstance(type);
        Console.WriteLine($"\nЗагружен плагин: {attribute.Name} ({type.Name})");

        // Вызов методов плагина через рефлексию
        foreach (MethodInfo method in type.GetMethods(BindingFlags.Public |
BindingFlags.Instance | BindingFlags.DeclaredOnly))
        {
            PluginMethodAttribute methodAttribute =
method.GetCustomAttribute<PluginMethodAttribute>();
            if (methodAttribute == null)
                continue;

            Console.WriteLine($"  Метод {method.Name}: {methodAttribute.Description}");
            if (method.Name == "Add")
                Console.WriteLine($"    Результат Add(5, 3) = {method.Invoke(plugin, new object[]
{ 5, 3 })}");
            else
                method.Invoke(plugin, null);
        }
    }
}
```